

Biomedical Data Quality System

System Architecture, Database Schema and Quality Metrics Justification

Daria Bartkowiak

May 9, 2026

Contents

1	GitHub	2
2	System Architecture	2
2.1	General Architecture	2
2.2	User Interface Layer	2
2.3	Reactive Application Layer	3
2.4	Persistent SQL Database Layer	3
3	Database Schema	4
3.1	Admissions Table	4
3.2	Primary Key	4
3.3	Audit Log Table	4
4	Quality Metrics Justification	5
4.1	Overview	5
4.2	Completeness	6
4.3	Consistency	6
4.4	Accuracy and Clinical Outlier Detection	6
5	Defensive Validation	7
6	Data Visualization and Exploration	8
7	Automated Report Generation	8

1 GitHub

<https://github.com/bartkowiakdaria/DSInMedicineFinalProject>

2 System Architecture

2.1 General Architecture

The application follows a modular reactive architecture implemented using the R **Shiny** framework. The system is divided into independent layers, each responsible for a specific part of the workflow.

The architecture consists of four main layers:

1. User Interface Layer,
2. Reactive Application Layer,
3. Data Quality and Validation Layer,
4. Persistent SQL Database Layer.

This separation improves maintainability, readability, and future extensibility of the system.

2.2 User Interface Layer

The user interface was implemented using **shiny** and **bslib**. It provides a navigation-based layout with several independent modules:

- Dashboard,
- Data Visualization,
- Record Management,
- Add Record,
- Reports,
- Audit Logs.

Each navigation tab corresponds to a separate functional module. This structure allows fluid navigation between data visualization, record management, and quality indicators.

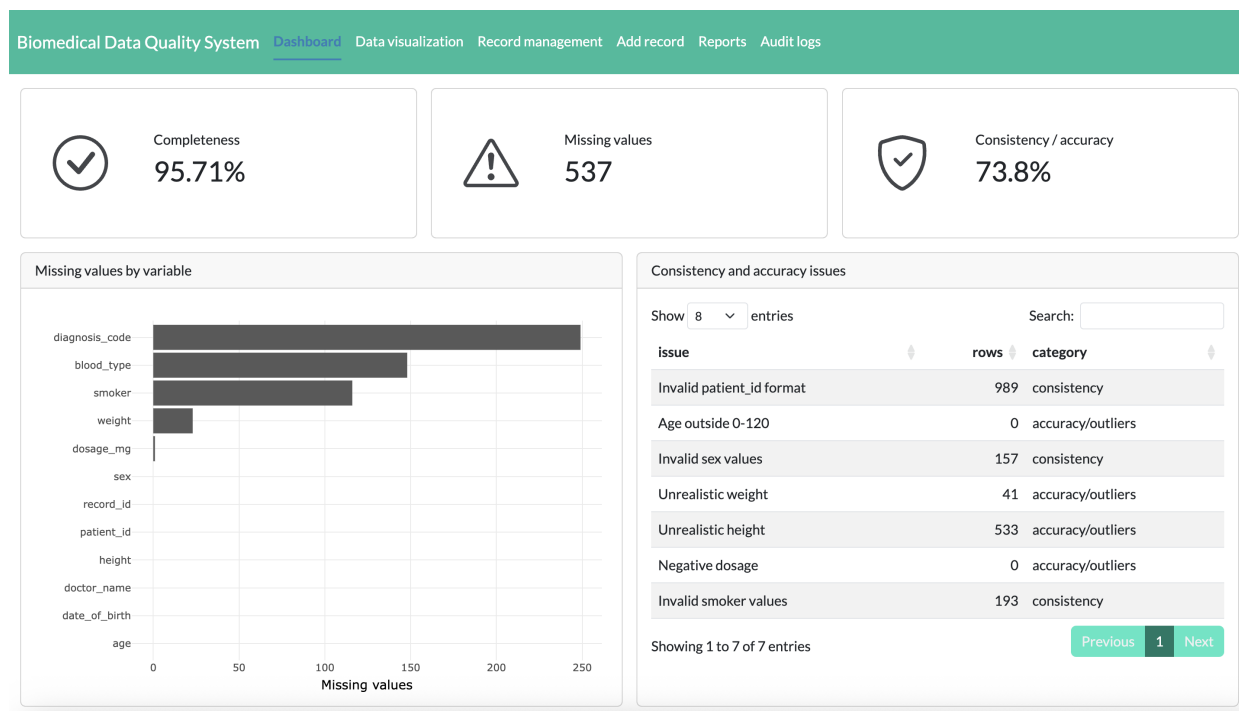


Figure 1: Dashboard presenting completeness, consistency, and accuracy indicators.

2.3 Reactive Application Layer

The server side of the application uses Shiny reactive programming. Reactive values are used to store the current database state, while reactive expressions automatically update quality indicators and visualizations.

The reactive architecture enables:

- automatic dashboard updates after database changes,
- dynamic generation of tables and plots,
- synchronized interaction between modules,
- immediate feedback after record insertion or deletion.

The application uses modular server functions to separate responsibilities. This makes the code easier to maintain and supports future expansion.

2.4 Persistent SQL Database Layer

The backend was implemented using SQLite through the DBI and RSQLite packages. SQLite was selected because it is lightweight, serverless, easy to deploy, and suitable for local biomedical data management prototypes.

At application startup, the system:

1. checks whether the database exists,
2. imports the initial CSV file,
3. standardizes column names,
4. creates the admissions table,
5. creates the audit log table.

3 Database Schema

3.1 Admissions Table

The main database table **admissions** stores biomedical and patient-related information imported from the CSV dataset.

The table contains variables related to:

- patient identity,
- demographic information,
- physiological measurements,
- diagnosis,
- medication,
- record metadata.

Important fields include:

Variable	Description
<code>record_id</code>	Unique internal record identifier.
<code>patient_id</code>	Patient identifier used for record tracking.
<code>date_of_birth</code>	Patient date of birth.
<code>age</code>	Recorded patient age.
<code>computed_age</code>	Age calculated from date of birth.
<code>sex</code>	Patient sex or gender category.
<code>weight_kg</code>	Patient weight in kilograms.
<code>height_cm</code>	Patient height in centimeters.
<code>blood_type</code>	Patient blood group.
<code>diagnosis_code</code>	Clinical diagnosis code.
<code>dosage_mg</code>	Medication dosage in milligrams.
<code>doctor_name</code>	Responsible physician.
<code>created_by</code>	User who created the record.
<code>created_at</code>	Timestamp of record creation.

Table 1: Main fields stored in the admissions table.

3.2 Primary Key

The variable `record_id` acts as the primary key of the admissions table. It is generated automatically when records are inserted.

3.3 Audit Log Table

The second table is `audit_log`. It stores information about operations performed in the application.

Variable	Description
id	Unique audit log entry identifier.
timestamp	Exact time when the operation occurred.
action	Type of operation, for example CREATE or DELETE.
record_id	Identifier of the affected patient record.
user	User responsible for the operation.
details	Additional description of the performed action.

Table 2: Fields stored in the audit log table.

The audit log table supports biomedical traceability by recording who modified the database, when the action occurred, and which record was affected.

Biomedical Data Quality System Dashboard Data visualization Record management Add record Reports Audit logs					
Audit logs					
Show 10 entries		Search:			
id	timestamp	action	record_id	user	details
1	2026-05-07 20:09:06.930389	CREATE	1043	unknown_user	New record added by unknown_user
2	2026-05-09 14:38:28.575202	CREATE	1044	unknown_user	New record added by unknown_user
3	2026-05-09 14:39:11.106099	CREATE	1045	unknown_user	New record added by unknown_user
Showing 1 to 3 of 3 entries					
		Previous 1 Next			

Figure 2: Audit log module showing database operations and timestamps.

4 Quality Metrics Justification

4.1 Overview

Biomedical databases are vulnerable to missing values, inconsistent entries, invalid categories, and clinically implausible measurements. To address these problems, the system automatically calculates three categories of quality metrics:

1. Completeness,
2. Consistency,
3. Accuracy.

These metrics are recalculated whenever the database changes.

4.2 Completeness

Completeness measures how much information is available in the database. It is calculated as the percentage of non-missing cells:

$$Completeness = 100 \times \left(1 - \frac{MissingCells}{TotalCells}\right)$$

This metric is important because missing clinical data can reduce statistical power, introduce bias, and limit the reliability of biomedical conclusions.

The application calculates:

- total number of missing cells,
- missing values per variable,
- global completeness percentage.

The dashboard also visualizes missing values by variable, allowing users to identify fields requiring attention.

4.3 Consistency

Consistency evaluates whether values are logically coherent across variables.

Implemented consistency rules include:

- checking whether `age` and `computed_age` are coherent,
- verifying the format of `patient_id`,
- validating categorical values such as `sex`,
- validating smoker status encoding.

These checks were selected because biomedical datasets often contain manual entry errors, formatting problems, or contradictory values. For example, a mismatch between recorded age and computed age may indicate an incorrect date of birth or incorrect manual entry.

Consistency metrics help detect records that require manual review before further analysis.

4.4 Accuracy and Clinical Outlier Detection

Accuracy refers to whether recorded values are clinically plausible. The system identifies potential clinical outliers using predefined biomedical rules.

Implemented accuracy checks include:

- age outside the range 0–120 years,
- body weight outside realistic human ranges,
- body height outside realistic human ranges,
- negative medication dosage.

These rules were selected because extreme or impossible clinical values often indicate data-entry mistakes, incorrect units, measurement errors, or corrupted records.

5 Defensive Validation

The data entry form includes defensive validation to prevent erroneous records from being saved into the database.

The system validates:

- mandatory patient identifier,
- patient identifier format,
- mandatory age,
- logical age range,
- mandatory sex value,
- valid body weight,
- valid body height,
- non-negative dosage,
- mandatory doctor name,
- mandatory user identifier.

If validation fails, the application blocks submission and displays visual alerts. This prevents invalid biomedical records from entering the persistent SQL database.

The screenshot shows a web application interface for a Biomedical Data Quality System. The top navigation bar is green and contains links: Dashboard, Data visualization, Record management, Add record (highlighted), Reports, and Audit logs. Below the navigation bar is a form titled "Add new admission record with defensive validation". The form contains several input fields: patient_id (with the value "uo98"), date_of_birth, age, sex (a dropdown menu), weight, height, blood_type, and diagnosis_code. A red alert box in the bottom right corner of the form displays the following error messages: "patient_id must have format like P1001. age is mandatory and must be numeric. sex is mandatory. doctor_name is mandatory." The alert box has a close button (X) in the top right corner.

Figure 3: Defensive validation preventing submission of invalid biomedical data.

6 Data Visualization and Exploration

The visualization module supports exploratory analysis of biomedical variables. It uses `ggplot2` and `plotly` to create interactive plots.

Supported visualizations include:

- histograms for numeric variables,
- bar charts for categorical variables,
- scatter plots for relationships between numeric variables.

The module automatically adapts to the selected variable type. If a non-numeric variable is selected for a histogram, the system displays a bar chart instead.

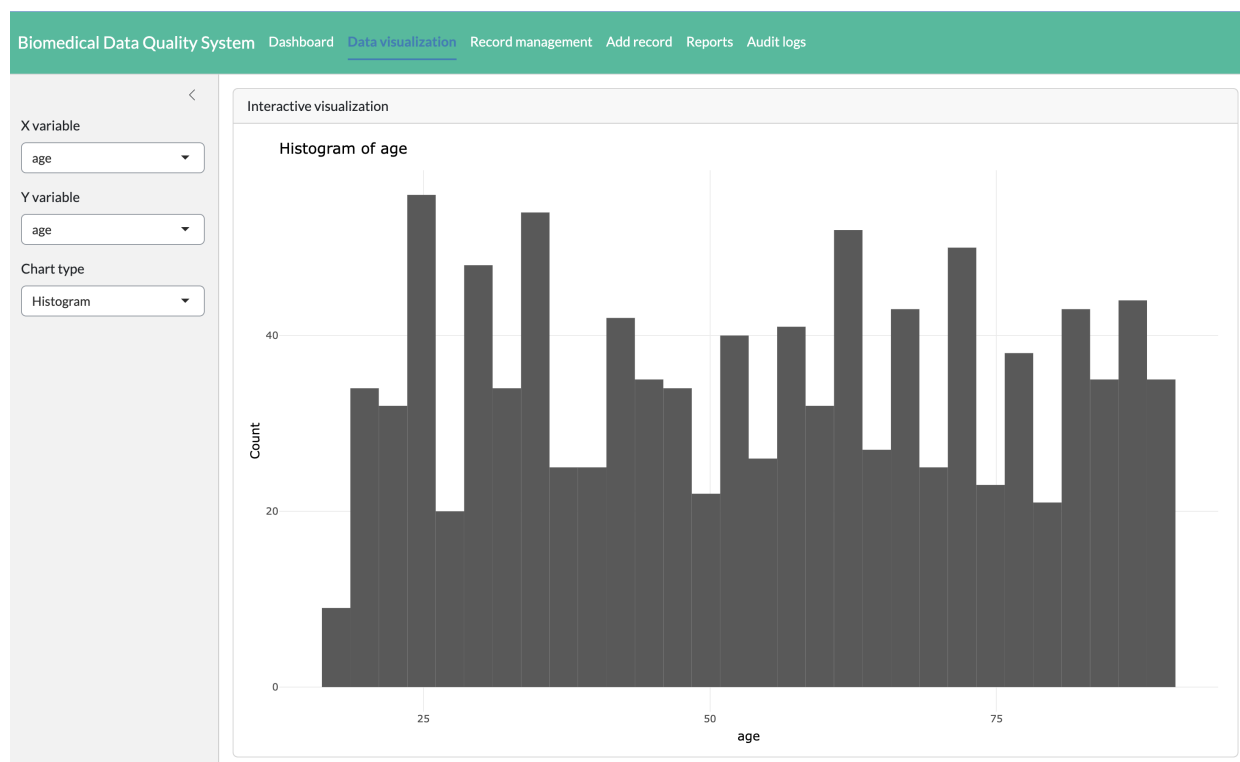


Figure 4: Interactive visualization module for biomedical data exploration.

7 Automated Report Generation

The application includes automated report generation using `RMarkdown`. Reports can be downloaded directly from the Shiny interface in HTML format.

The report includes:

- database dimensions,
- completeness metrics,
- missing values by variable,
- consistency and accuracy issues,
- audit logs.